DBSCAN
& Hierarchical
Clustering

Density-Based · Hierarchies · Dendrograms · Any-Shape Clusters · No K Needed

DBSCAN | ε & minPts | Core Points | Dendrogram | Ward Linkage | Agglomerative

What We'll Cover Today

01 **Why K-Means Isn't Enough**
The shapes and noise problems

02 **DBSCAN Core Concepts**
ε · minPts · Core / Border / Noise

03 **DBSCAN Algorithm & Math**
Step-by-step + density reachability

04 **DBSCAN Numerical Example**
Run DBSCAN by hand on real data

05 **DBSCAN Parameters & Tips**
Choosing ε and minPts correctly

06 **DBSCAN vs K-Means**
Full head-to-head comparison

07 **Hierarchical Clustering Intro**
Agglomerative vs Divisive

08 **The Dendrogram Explained**
How to read & cut a dendrogram

09 **Linkage Methods Deep Dive**
Single · Complete · Average · Ward

10 **Hierarchical Numerical Example**
Trace the full merge sequence

11 **Python Code — Both Methods**
sklearn implementation + visualization

12 **Choosing Your Algorithm**
Decision guide: when to use which

Why K-Means Isn't Enough — The Motivation

K-Means makes strong assumptions that fail in real data. DBSCAN and Hierarchical clustering solve these problems.

Non-Spherical Clusters

K-Means assumes clusters are round blobs. Real data has spirals, crescents, rings, elongated shapes. K-Means will incorrectly split any non-convex cluster into pieces.

📌 Two interlocking crescents → K-Means always fails. DBSCAN finds them perfectly.

Noise & Outliers

K-Means forces EVERY point into a cluster — even obvious outliers. One extreme outlier pulls the centroid away from the true cluster center, corrupting it.

📌 A network log with 99% normal traffic and 1% attacks → K-Means cannot separate them.

Must Specify K

K-Means requires you to choose K before running. Wrong K = meaningless clusters. There's no way to know K without running the algorithm multiple times.

📌 Customer data may have 3 or 7 or 12 natural segments. How do you know?

Equal-Density Assumption

K-Means creates similarly-sized clusters. Real clusters have very different densities: a dense city center vs. sparse suburban area would be merged into one cluster.

📌 Geographic data: DBSCAN naturally finds dense urban clusters + sparse rural areas.

01

DBSCAN

Density-Based Spatial Clustering of Applications with Noise

Finds clusters of ANY shape · Labels outliers · No K required

DBSCAN — Core Concepts

Two parameters define everything: ε (epsilon) controls neighborhood size, minPts controls density threshold.

Analogy: Think of a city at night seen from above. Dense neighborhoods are clusters. Isolated houses in the countryside are noise. DBSCAN finds the neighborhoods by local density — not by distance to a center.

ε — Epsilon (Neighborhood Radius)

The radius around each point that defines its 'neighborhood'. Any point within ε distance is a neighbor. You set this — it controls how tight clusters need to be.

💡 Tip: Use k-NN distance plot to find a good ε automatically.

minPts — Minimum Points

Minimum number of points (including the point itself) within ε distance to classify a point as a 'core point'. Controls minimum density to form a cluster.

💡 Rule of thumb: minPts ≥ d+1 where d = number of dimensions. For 2D: minPts=3.

Core — Core Point

A point that has at least minPts neighbors within its ε-neighborhood. Core points are the 'dense interior' of clusters. They can expand the cluster to their neighbors.

💡 Core points are always assigned to a cluster — never labeled as noise.

Border — Border Point

Has fewer than minPts neighbors within ε, but is within ε of a Core point. Sits on the cluster edge. Belongs to the cluster but cannot expand it further.

💡 Border points near multiple clusters are assigned to the first cluster found.

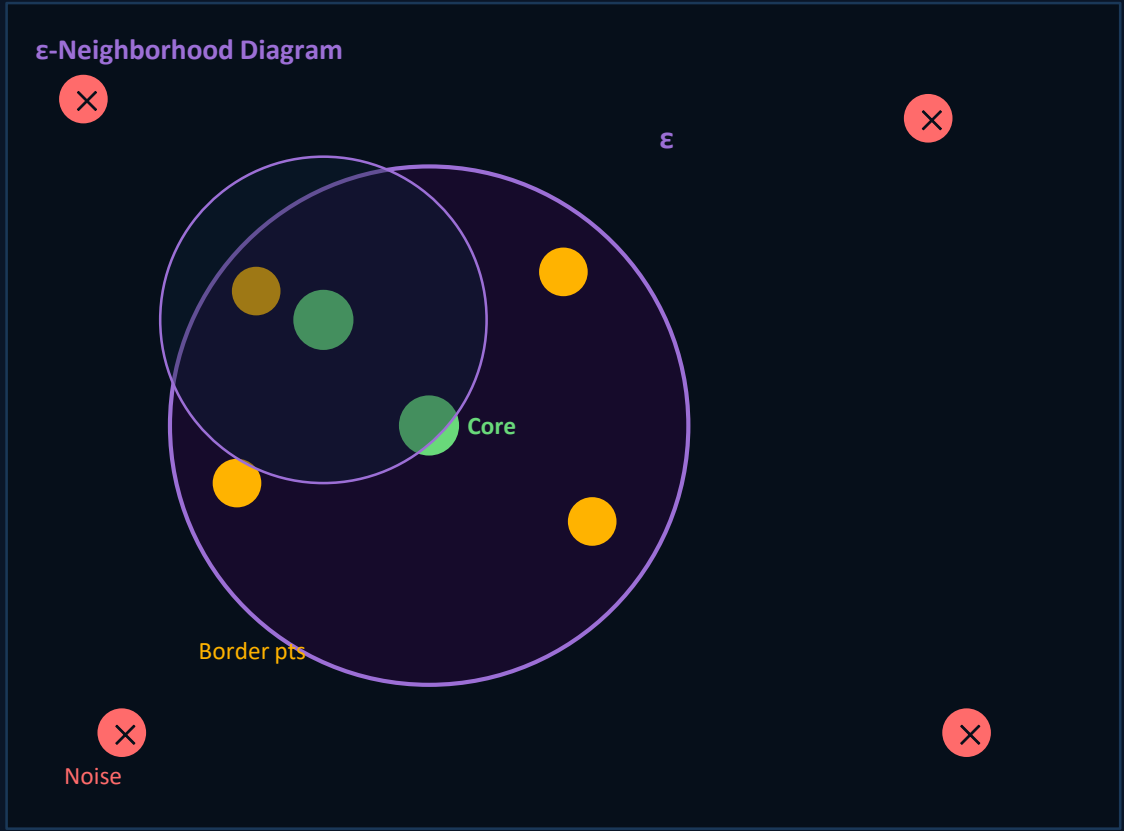
Noise −1 — Noise / Outlier

Not a Core point AND not within ε of any Core point. Labeled −1 in sklearn. These are genuine outliers — DBSCAN automatically separates them, which K-Means cannot do.

💡 Key DBSCAN superpower: outlier detection comes for free!

DBSCAN — Visualizing Point Types

The three point types shown for a cluster with ε=1.0, minPts=4. Each colored circle shows a neighborhood.



Core point · Border point · Noise (−1) · ε-neighborhood

Directly Density-Reachable

Point p is DDR from q if p is in ε(q) and q is a core point. Captures the 'neighbor' relationship.

Density-Reachable

p is DR from q if there is a chain $p_1 \to p_2 \to \ldots \to p_n$ where each step is DDR. Captures connected paths through dense areas.

Density-Connected

p and q are DC if there exists a point o such that both p and q are density-reachable from o. Defines the cluster relationship.

Cluster Definition

A cluster C is a maximal set of density-connected points. Every point in C is density-reachable from every core point in C.

DBSCAN Algorithm — Complete Step-by-Step

Time complexity: O(n log n) with spatial index (k-d tree). O(n²) naive. Space: O(n).

1 Mark all as Unvisited

Initially every point is marked UNVISITED. No cluster assignments yet. Create a visited[] array and labels[] array, all empty.

2 Pick unvisited point p

Take any unvisited point p. Mark it as visited. This prevents re-processing and ensures we visit every point exactly once.

3 Find ε-neighborhood

Compute all points within ε distance of p: $N(p) = \{q \mid dist(p,q) \le \varepsilon\}$. This is the query step — typically uses a k-d tree for speed.

4 |N(p)| < minPts?

If the neighborhood is too small: mark p as NOISE (label −1). But note: a noise point can later become a border point if a core point reaches it.

5 p is a Core Point!

If $|N(p)| \ge minPts$: create new cluster C. Add p to C. Now expand the cluster by exploring all neighbors of p.

6 Expand cluster

For each neighbor q in N(p): if q is unvisited, visit it. If $|N(q)| \ge minPts$, add N(q) to the expansion queue. Add q to cluster C.

7 Repeat until done

Continue expanding until no more unvisited points in the queue. Then return to step 2 and pick the next unvisited point from the dataset.

8 All visited → done!

Algorithm terminates when every point has been visited. Result: each point has a label (0,1,2… for clusters or −1 for noise).

Euclidean distance (default): $dist(p,q) = \sqrt{\sum_j (p_j - q_j)^2}$ Works with any distance metric: Manhattan, Cosine, Haversine (GPS) — set metric= in sklearn

DBSCAN Numerical Example — Run by Hand

Dataset: 8 points in 2D. Parameters: ε = 2.0, minPts = 3.

Point	x	y	Neighbors within ε=2	|N|	≥ minPts?	Type	Cluster
P1	1	1	P2, P3, P8	3	YES ✓	Core	C1
P2	1.5	1.5	P1, P3, P8	3	YES ✓	Core	C1
P3	2	1	P1, P2, P8	3	YES ✓	Core	C1
P8	1.8	0.8	P1, P2, P3	3	YES ✓	Core	C1
P4	5	5	P5, P6	2	NO ✗	Border	C2
P5	5.5	5.5	P4, P6	2	NO ✗	Core	C2
P6	6	5	P4, P5	2	NO ✗	Border	C2
P7	9	9	(none)	0	NO ✗	NOISE	−1

Result: Cluster 1 = {P1, P2, P3, P8} Cluster 2 = {P4, P5, P6} Noise = {P7 labeled −1} Two dense groups discovered automatically!

Choosing ε and minPts — The Science Behind the Parameters

Wrong parameters = wrong clusters. Here's how to choose them correctly.

Rule for minPts: minPts ≥ d + 1 (where d = number of features). For 2D data: use 3 or 4. For higher dimensions (d≥10): use 2×d or more. Larger minPts = more robust to noise but may miss small clusters.

Finding ε — The k-NN Distance Plot

Algorithm:

1. Compute the k-th nearest neighbor distance for every point (k = minPts − 1)
2. Sort all distances in ascending order
3. Plot them — the curve has a characteristic 'knee'
4. The ε value at the knee is your optimal threshold

Python:

```
from sklearn.neighbors import NearestNeighbors
nbrs = NearestNeighbors(n_neighbors=4).fit(X)
dists, _ = nbrs.kneighbors(X)
plt.plot(sorted(dists[:,-1]))  # look for knee!
```

ε too SMALL

Most points become noise. Clusters are fragmented into many tiny pieces. Model is too strict — misses real structure.

ε just RIGHT

Natural clusters form. Noise points are genuine outliers. Core/border/noise split matches the real data structure.

ε too LARGE

Everything merges into 1 giant cluster. No structure detected. Model is too lenient — ignores all density variation.

Effect of minPts ↑

Higher minPts → stricter definition of 'dense' → more noise points → larger clusters with stronger cores.

DBSCAN vs K-Means — Complete Comparison

	K-Means	DBSCAN
Needs K?	YES — set before running	NO — found automatically ✅
Cluster shape	Spherical only	ANY shape ✅ (convex, ring, crescent)
Outlier handling	Forces all into clusters	Labels as noise −1 ✅
Equal density?	Assumes similar density	Handles varying density ✅
Training speed	O(n·K·iter) — very fast	O(n log n) with k-d tree
Memory	O(K) — tiny	O(n) — stores neighbors
Deterministic?	No (depends on init)	Yes (same result every run) ✅
Feature scaling	Required	Required
High dimensions	Works well	Struggles (curse of dim.)
Interpretability	High (centroids are intuitive)	Medium (density concept)
Best for	Compact blobs, known K	Arbitrary shapes, unknown K, noisy data

02

Hierarchical Clustering

Build a tree of clusters — choose K AFTER seeing the full picture

Hierarchical Clustering — Two Opposite Strategies

Both create the same dendrogram but from opposite ends. Agglomerative is by far the most common.

Analogy: Agglomerative = biologists classifying species from individual animals up to kingdoms. Divisive = splitting a school into grades, then classes, then groups.

↑ AGGLOMERATIVE (Bottom-Up)

1. Start: n clusters (each point = 1 cluster)
2. Find the 2 closest clusters
3. Merge them into 1 (n−1 clusters now)
4. Repeat until 1 cluster remains (root)
5. Cut dendrogram at chosen height → K clusters

```
sklearn: AgglomerativeClustering(n_clusters=3, linkage='ward')
```

↓ DIVISIVE (Top-Down)

1. Start: 1 cluster (entire dataset)
2. Pick the cluster to split (highest diameter)
3. Apply 2-means or bisection to split it
4. Get 2 sub-clusters (total +1 cluster now)
5. Repeat until K clusters or all singletons

```
sklearn: BisectingKMeans(n_clusters=3)  — divisive approach
```

Linkage Methods — How Do We Measure Distance Between Clusters?

The linkage method defines what 'distance between two clusters' means. It is the most critical hyperparameter.

Single Linkage

$$d(A,B) = \min\{ dist(a,b) : a \in A, b \in B \}$$

Distance = minimum distance between ANY pair of points across the two clusters. The 'most optimistic' measure.

✓ Can find elongated, chain-like clusters. Handles non-convex shapes.

✗ Chaining effect: long chains form as clusters gradually merge. Very sensitive to noise and outliers.

Best for: When clusters have irregular shapes and no noise.

Complete Linkage

$$d(A,B) = \max\{ dist(a,b) : a \in A, b \in B \}$$

Distance = maximum distance between ANY pair. The 'most pessimistic' measure — two clusters are only close if ALL their points are close.

✓ Creates compact, tightly-bounded clusters. Very robust to outliers and noise.

✗ Biased toward creating clusters of similar diameter. Can split large natural clusters.

Best for: When you want compact, roughly equal-sized clusters.

Average Linkage (UPGMA)

$$d(A,B) = (1/|A||B|) \cdot \Sigma_{a \in A} \Sigma_{\beta \in B} dist(a,b)$$

Distance = average distance between ALL pairs of points across the two clusters. A compromise between Single and Complete.

✓ Balanced — less prone to chaining than Single, less biased than Complete. Works well in practice.

✗ More computationally expensive. Results depend heavily on the scale of the data.

Best for: General purpose — good first choice when unsure.

Ward Linkage

$$\Delta = SS_{a+\beta} - SS_a - SS_\beta \quad \text{(increase in within-cluster SS)}$$

Merges the two clusters that minimize the increase in total within-cluster variance (sum of squared distances to centroids). Minimizes inertia.

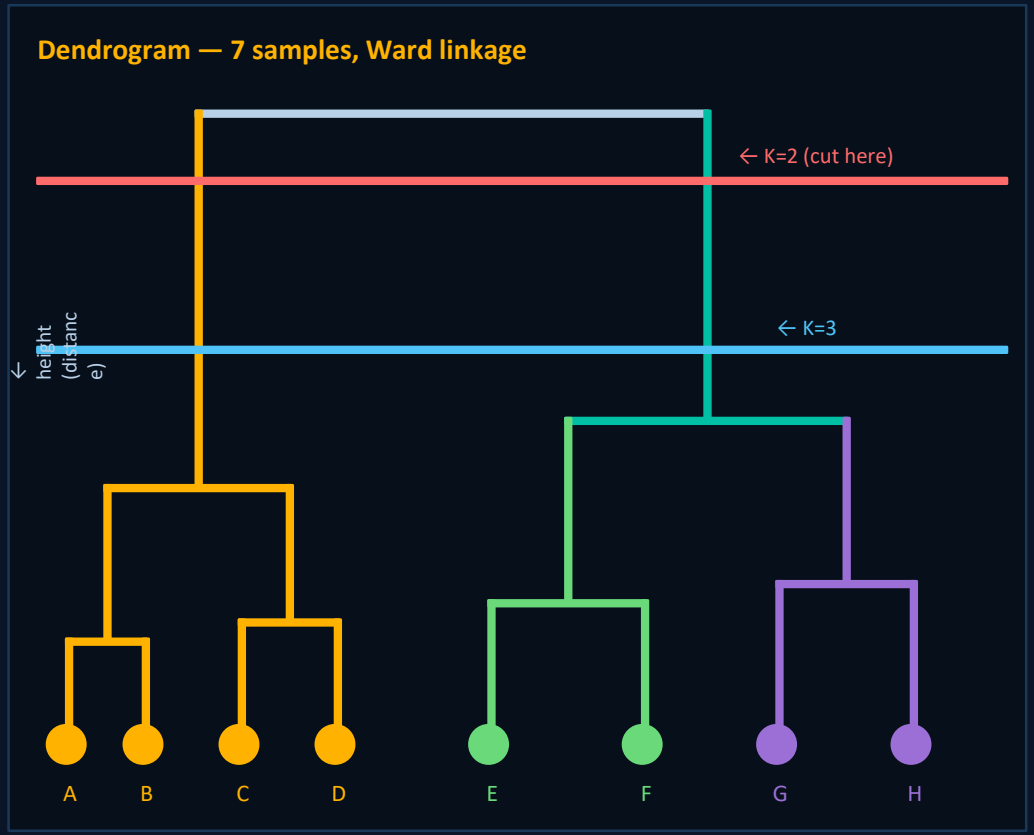
✓ Creates compact, equal-variance clusters. Often gives the best practical results. Most popular choice.

✗ Assumes spherical clusters. Only defined for Euclidean distance (not cosine or other metrics).

Best for: Default choice for most real-world tabular data problems.

The Dendrogram — Reading & Cutting the Tree

A dendrogram encodes the FULL clustering hierarchy. You choose K by where you 'cut' the tree.



Height = merge distance

The y-position of each horizontal bar = the distance (or dissimilarity) at which those clusters merged. Higher = more different.

Tall bars = natural cuts

Long vertical lines before the next merge mean clusters that were quite different when joined. These are the natural cutting points.

Cutting the tree

A horizontal cut at height h gives K clusters = number of vertical lines it crosses. Move the cut up → fewer clusters. Down → more clusters.

Cophenetic correlation

Measures how well the dendrogram preserves pairwise distances. >0.75 = good. Use to compare linkage methods for your data.

Hierarchical Clustering — Numerical Example (Single Linkage)

5 points in 1D. Trace every merge step. Distances: d(A,B)=1, d(A,C)=4, d(A,D)=6, d(A,E)=9, d(B,C)=3, d(B,D)=5, d(B,E)=8, d(C,D)=2, d(C,E)=5, d(D,E)=3.

Points: A=1, B=2, C=5, D=7, E=10 (1D data, Single linkage = minimum distance between clusters)

	A	B	C	D	E
A	0	1	4	6	9
B	1	0	3	5	8
C	4	3	0	2	5
D	6	5	2	0	3
E	9	8	5	3	0

Merge Heights (for dendrogram):

Step 1: A+B merged → height = 1

Step 2: C+D merged → height = 2

Step 3: {AB}+{CD} → height = 3

Step 4: +E (root) → height = 3

To get K=2 clusters: cut above height 2 → {A,B,C,D} and {E}

Step 1

Merge A + B

d(A,B)=1 is minimum in matrix

New cluster {AB}. Distance {AB}→C = min(d(A,C),d(B,C)) = min(4,3) = 3

Step 2

Merge C + D

d(C,D)=2 is new minimum

New cluster {CD}. Distance {AB}→{CD} = min(d(A,C),d(A,D),d(B,C),d(B,D)) = min(4,6,3,5) = 3

Step 3

Merge {AB}+{CD}

d({AB},{CD})=3 and d({CD},E)=3 tie! Take first.

New cluster {ABCD}. Distance {ABCD}→E = min(3,5) = 3 (single linkage tracks min)

Step 4

Merge {ABCD}+E

Only 2 clusters remain, d=3 → forced merge

Root cluster {ABCDE}. Dendrogram complete! Heights: 1,2,3,3

Python Implementation — DBSCAN & Hierarchical

```python
# == DBSCAN ================================
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler
import numpy as np

# Scale features — critical for DBSCAN!
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Find ε using k-NN distance plot
from sklearn.neighbors import NearestNeighbors
nbrs = NearestNeighbors(n_neighbors=4).fit(X_scaled)
dists, _ = nbrs.kneighbors(X_scaled)
# plot sorted(dists[:,-1]) and find the knee

# Train DBSCAN
db = DBSCAN(eps=0.5, min_samples=5)
labels = db.fit_predict(X_scaled)

# Analyse results
n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
n_noise = list(labels).count(-1)
print(f'Clusters: {n_clusters}')
print(f'Noise pts: {n_noise}')

# Core samples mask
core_mask = np.zeros_like(labels, dtype=bool)
core_mask[db.core_sample_indices_] = True
```

```python
# == HIERARCHICAL ==========================
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage
from scipy.spatial.distance import pdist

# Build full linkage matrix (for dendrogram)
Z = linkage(X_scaled, method='ward')
# Plot dendrogram
dendrogram(Z, truncate_mode='level', p=5)

# Train with chosen K
hc = AgglomerativeClustering(
    n_clusters=3,
    linkage='ward',   # best default
    metric='euclidean')
labels_hc = hc.fit_predict(X_scaled)

# Without specifying K — use distance threshold
hc_auto = AgglomerativeClustering(
    n_clusters=None,
    distance_threshold=5.0)
labels_auto = hc_auto.fit_predict(X_scaled)

# Cophenetic correlation — measure tree quality
from scipy.cluster.hierarchy import cophenet
c, _ = cophenet(Z, pdist(X_scaled))
print(f'Cophenetic r: {c:.3f}')  # want > 0.75
```

`pip install scikit-learn scipy` | Both use StandardScaler first | DBSCAN labels_ has −1 for noise | Ward linkage needs Euclidean distance

Algorithm Decision Guide

K-Means vs DBSCAN vs Hierarchical — which fits YOUR problem?

	K-Means	DBSCAN	Hierarchical
Needs K?	YES (before run)	NO — automatic ✅	After (cut dendro)
Cluster shape	Spherical only	ANY shape ✅	ANY shape ✅
Handles noise	No — forces all in	YES → labels −1 ✅	No
Time complexity	O(n·K·iter) fast	O(n log n) moderate	O(n^2 log n) slow
Space complexity	O(K) tiny	O(n)	O(n^2) — distance matrix
Deterministic?	No (init varies)	Yes ✅	Yes ✅
Varying density?	No	Struggles if very different	No
Interpretability	High (centroids)	Medium	Very High ✅ (tree)
Best data size	Any (millions OK)	Medium (<100k)	Small-Medium (<10k)
Need scaling?	YES	YES	YES
Best use case	Known K, compact blobs	Unknown K, noise, shapes	Hierarchy needed, small data

Session Summary — Key Takeaways

01

DBSCAN needs no K

Two parameters: ε (neighborhood radius) and minPts (density threshold). Finds clusters of ANY shape. Automatically labels outliers as −1.

02

3 point types in DBSCAN

Core points (dense centers) · Border points (cluster edges) · Noise points (genuine outliers). Only core points expand clusters.

03

Choose ε with k-NN plot

Plot the k-th nearest neighbor distances sorted. The 'knee' in the curve is your optimal ε. minPts = d+1 for d-dimensional data.

04

Hierarchical builds a tree

Agglomerative merges bottom-up. Each merge recorded in a dendrogram. You choose K AFTER seeing the full picture — maximum flexibility.

05

Ward linkage is the default

Minimizes within-cluster variance at each merge. Creates compact equal-size clusters. Best for most tabular data. Only works with Euclidean.

06

Tall dendrogram bars = K cuts

Long vertical lines before a merge = natural cluster boundary. Cut there. Cophenetic correlation > 0.75 = good dendrogram quality.

Great work!

You now master DBSCAN & Hierarchical Clustering.